10/22/2026

LAB NO 07

Operating System Lab

Instructor: Ms. Muneeba Banaras

Submitted By: Wasif Ali

Reg ID: 245187

Class: BSCS F'24 B

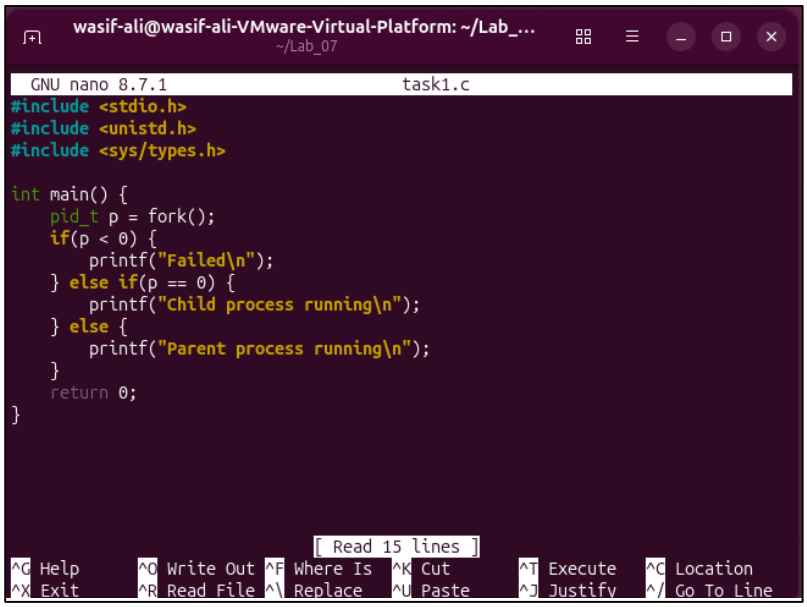
LAB TASK

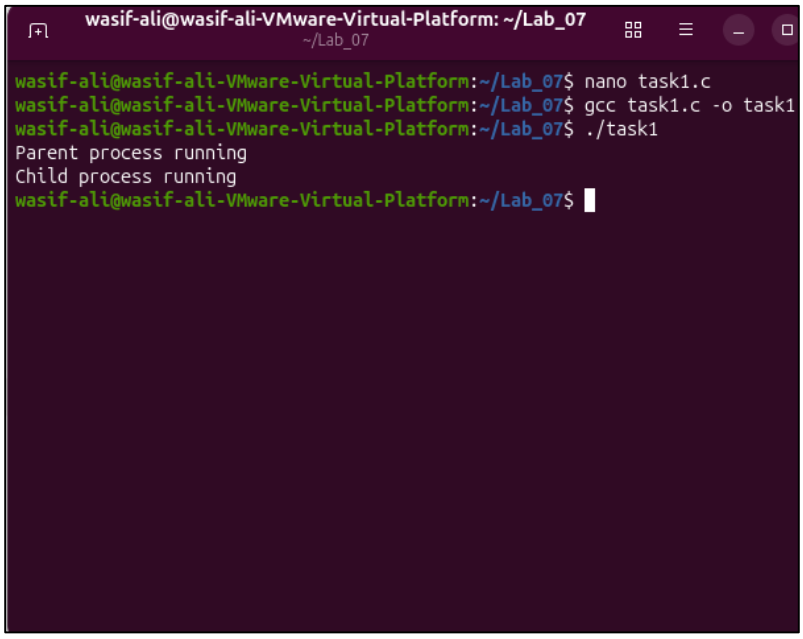
Lab Activity 1: Process Creation Using fork()

Problem Statement:

Solve the process creation problem by writing a C program that uses the fork() system call to create a child process. Show separate messages from the parent and child processes and demonstrate how both processes continue execution after the fork() statement.

Code/Commands:





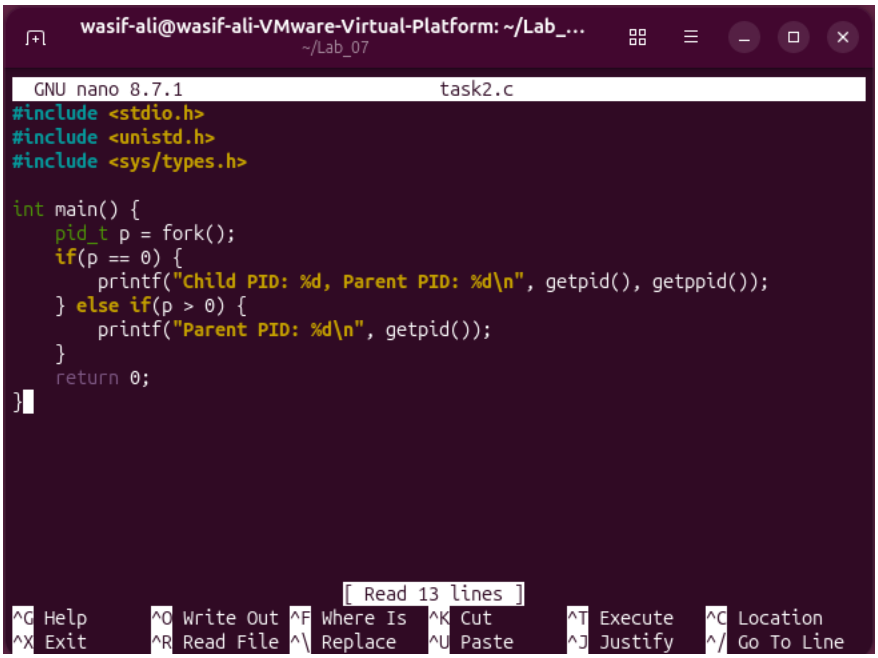
In this activity, we used the fork() function to create a new child process. We wrote a C program that prints separate messages to show that both the parent and the newly created child processes execute at the same time.

Lab Activity 2: Identify Parent and Child Processes

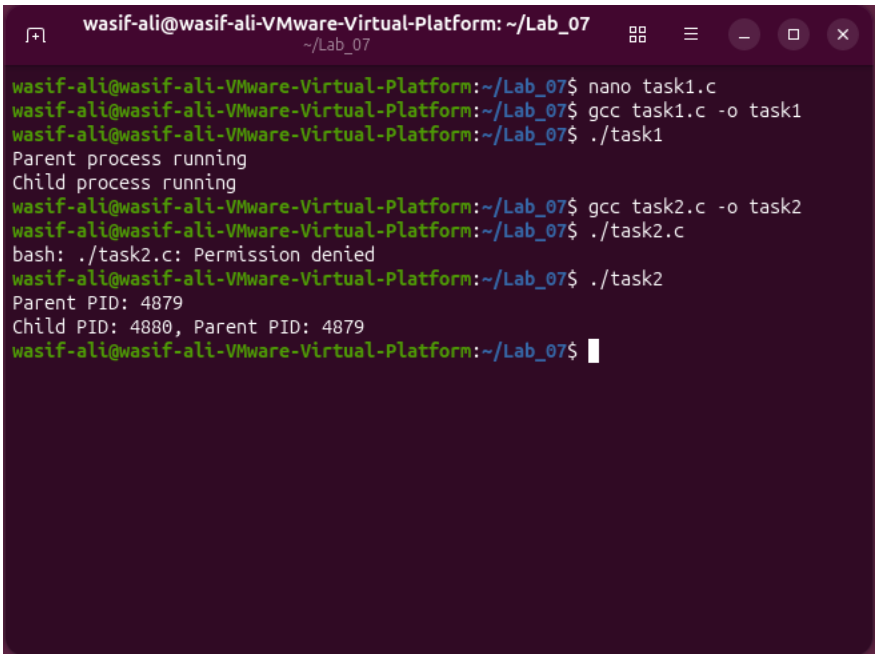
Problem Statement:

Use the getpid() and getppid() functions to identify the process ID and parent process ID of the parent and child processes. Relate the output of these functions to the process hierarchy and report the IDs displayed during program execution.

Code/Commands:



Here, we used the getpid() and getppid() functions to identify our processes. This allowed us to find and display the unique Process ID (PID) of the current process, as well as the PID of its parent process.

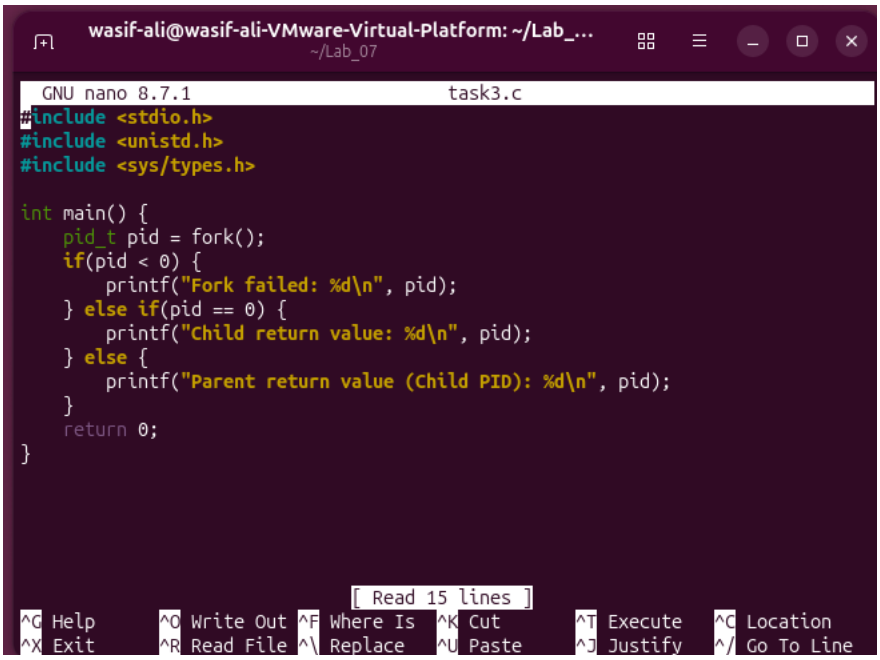


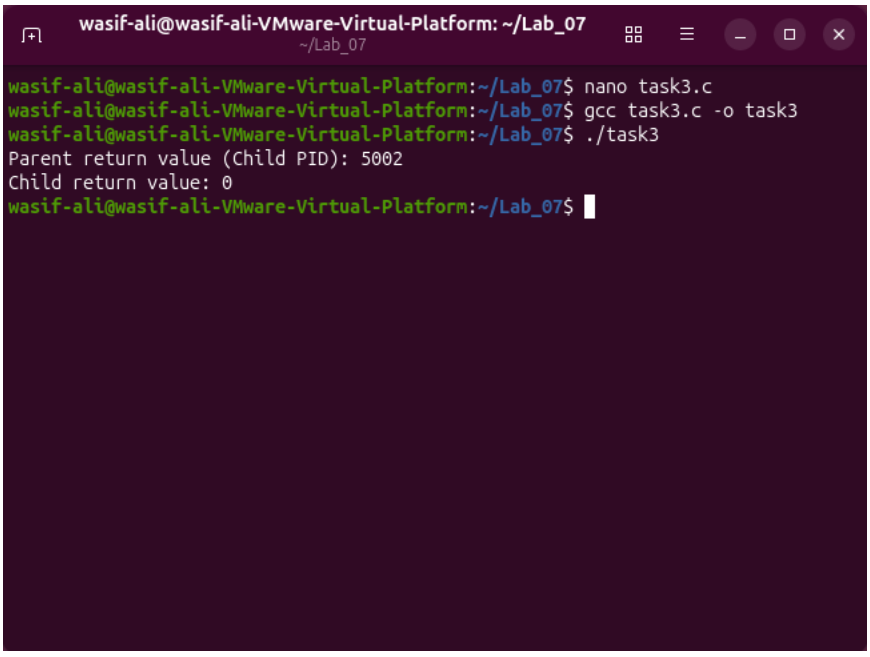
Lab Activity 3: Analyze the Return Value of fork()

Problem Statement:

Complete a C program that stores the return value of fork() in a variable of type pid_t. Change the program to handle the three possible return values: negative for failure, zero for the child process, and positive for the parent process. Articulate the purpose of each condition and show the corresponding output.

Code/Commands:





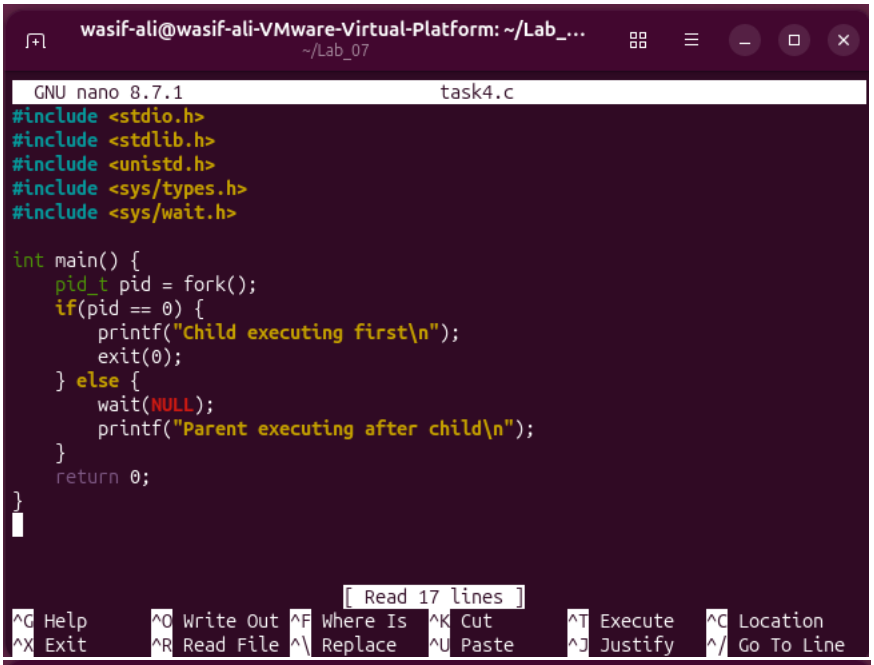
In this task, we checked the number returned by the fork() function. We used if-else conditions to print different messages based on whether the return value was negative (process creation failed), zero (we are in the child process), or positive (we are in the parent process).

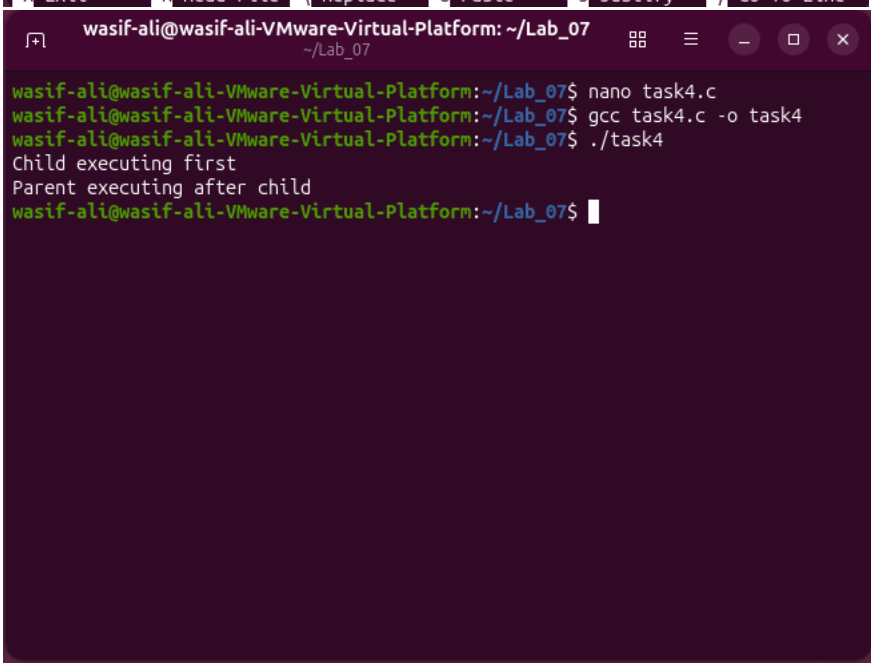
Lab Activity 4: Synchronize Processes Using wait()

Problem Statement:

Prepare a C program in which the parent process creates a child process and uses wait() to wait for the child to terminate. Demonstrate the difference between execution without synchronization and execution with synchronization. Discover how wait() affects the order of the displayed messages.

Code/Commands:





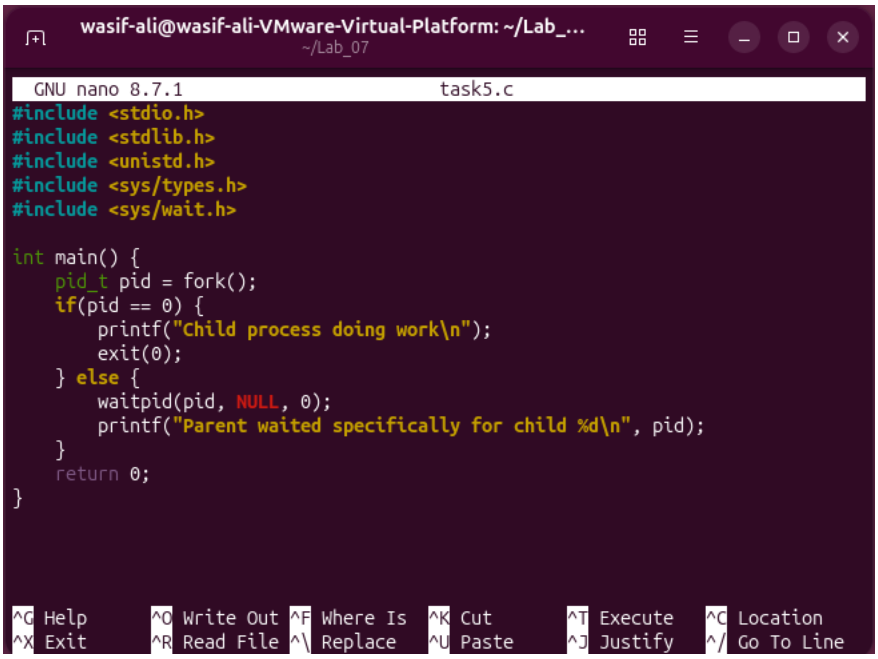
We learned how to synchronize processes using the wait() function. We made the parent process pause and wait for the child process to finish its execution completely before the parent continued with its own remaining code.

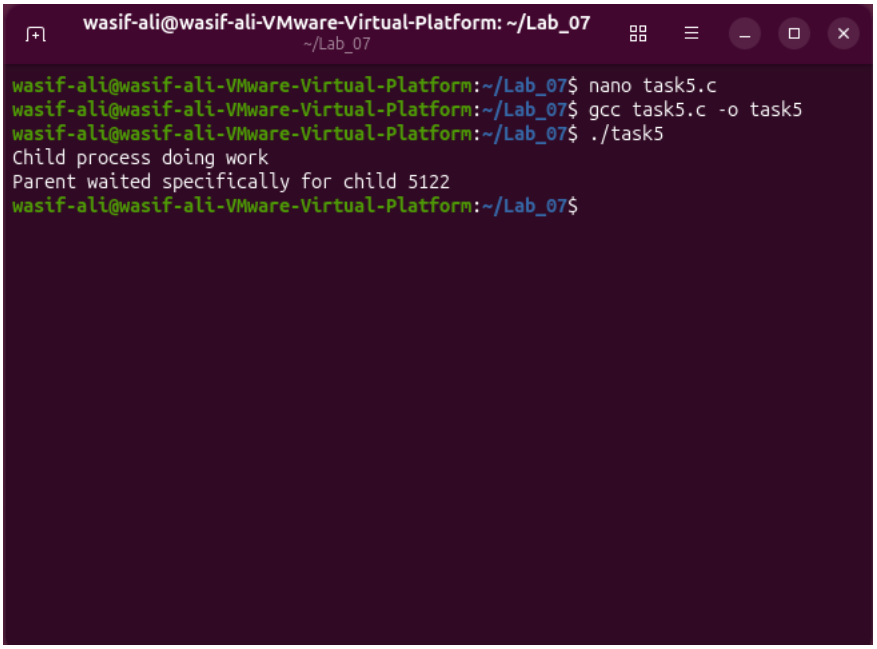
Lab Activity 5: Wait for a Specific Child Using waitpid()

Problem Statement:

Produce a C program that creates a child process and uses waitpid() to wait for that specific child. Manipulate the program by changing the child PID argument and explain how waitpid() differs from wait().

Code/Commands:





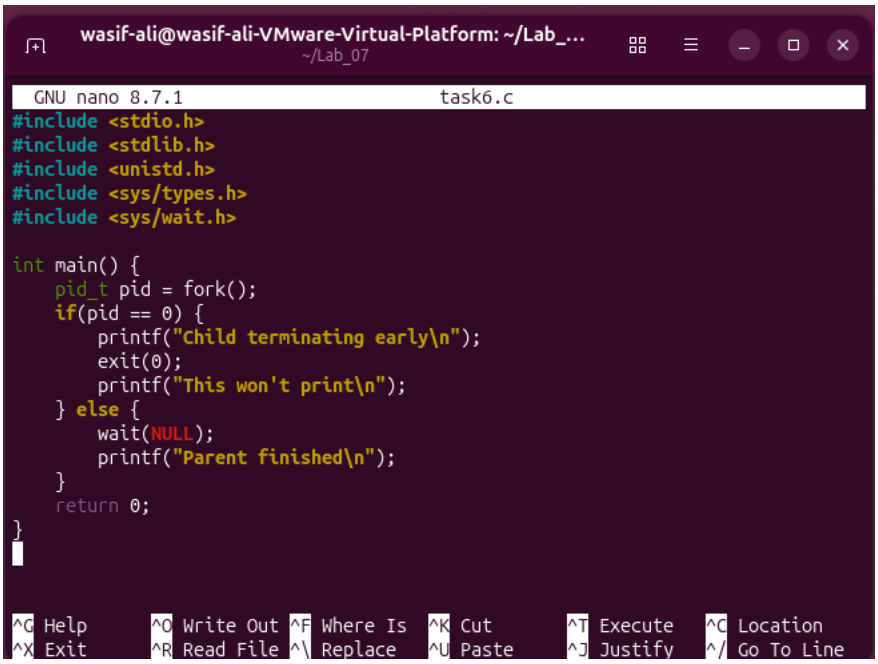
Building on the previous task, we used the waitpid() function instead of standard wait(). This gave us more control, allowing the parent process to wait for a *specific* child process to finish by targeting its exact Process ID.

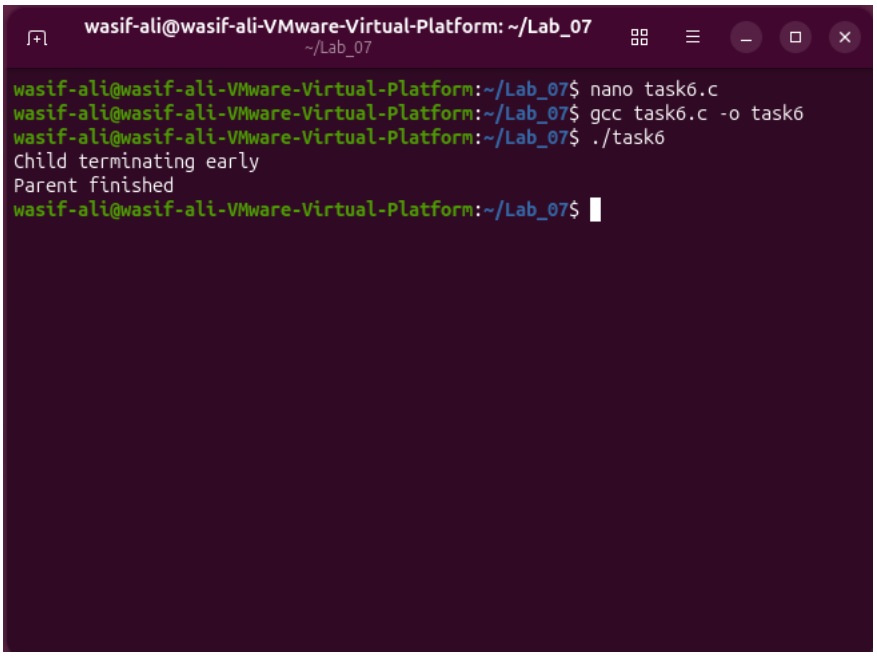
Lab Activity 6: Process Termination Using exit()

Problem Statement:

Demonstrate process termination by using exit(0) in the child process after displaying a message. Transfer the concept of successful termination to a program in which the parent waits for the child. Show that the parent continues after the child terminates, and explain the meaning of the exit status.

Code/Commands:





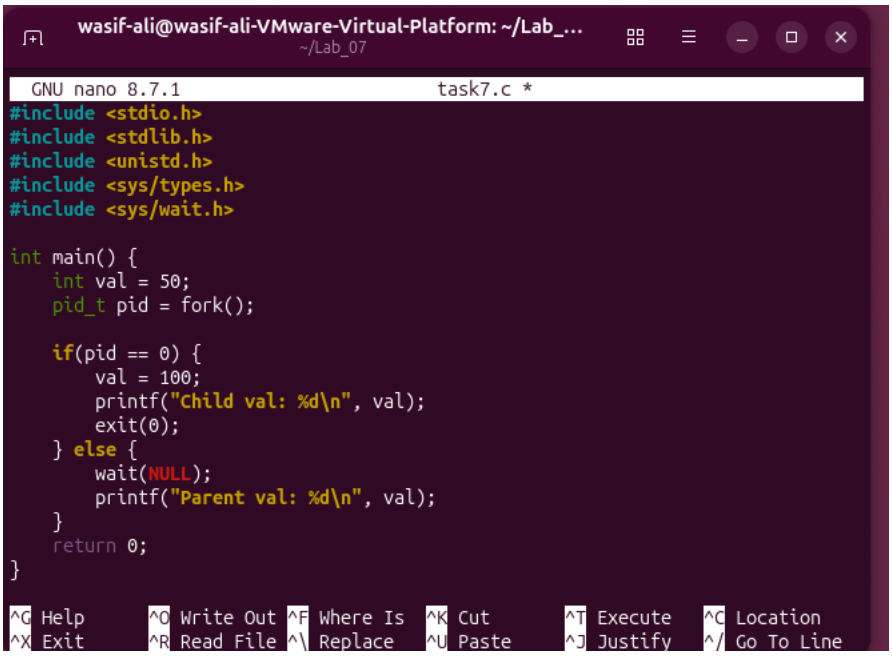
In this activity, we used the exit(0) function to safely terminate the child process. We demonstrated that when a process calls exit(), it stops immediately, and any lines of code written after the exit() command do not get executed.

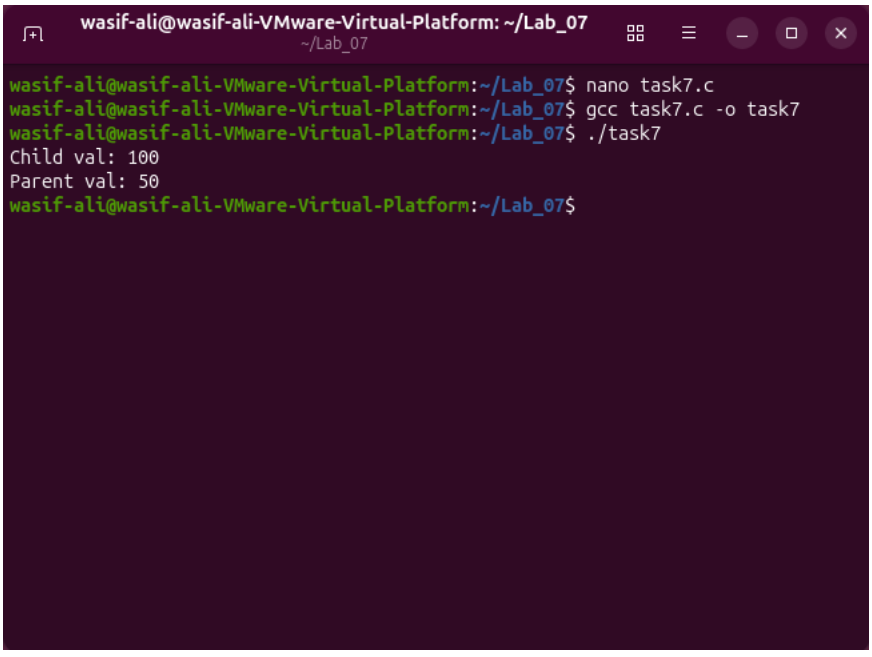
Lab Activity 7: Independent Variables in Parent and Child

Problem Statement:

Sketch the execution flow of a program that initializes an integer variable before calling fork(). Change the variable's value in the child process and show the values displayed by both processes. Relate the results to the separate memory spaces of parent and child processes.

Code/Commands:





In the final task, we proved that parent and child processes have their own separate memory spaces. We changed the value of a variable inside the child process and saw that the parent's copy of that same variable remained completely unchanged.